

Step-by-Step Calculation of Attention Mechanism

Sentence and Tokenization

Consider the sentence: "The bank of the river was flooded!"

item Tokens: [CLS], The, bank, of, the, river, was, flooded, [SEP]

The examples have 3 dimensions but DistilBert has 768 per token

Assume the following simplified embeddings for each token:

Creating Embedding Vectors

$$[\text{CLS}] = [1.0, 0.2, 0.1]$$

$$\text{The} = [0.2, 0.8, 0.1]$$

$$\text{bank} = [0.2, 0.1, 0.9]$$

$$\text{of} = [0.9, 1.0, 0.2]$$

$$\text{the} = [0.2, 0.8, 0.1]$$

$$\text{river} = [0.0, 0.9, 0.9]$$

$$\text{was} = [0.3, 0.8, 0.9]$$

$$\text{flooded} = [0.1, 0.9, 0.8]$$

$$[\text{SEP}] = [0.0, 0.3, 0.1]$$

$$W_Q, W_K, W_V$$

- These matrices are consistent across all inputs and represent weight matrices that were learned during backpropagation. These matrices are part of the model parameters and are optimized during training.

In BERT and DistilBert these weight matrices are of size:

$$W_Q = [768 \times 64]$$

$$W_K = [768 \times 64]$$

$$W_V = [768 \times 64]$$

- For simplicity our weight matrices will be of size 3 x 3
- For simplicity assume that the parameters(numbers) have already been optimized.

$$W_Q = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 15.33 & 0 \\ 0 & 0 & -13.11 \end{bmatrix}$$

$$W_K = \begin{bmatrix} 0.1 & 0.2 & 0.3 \\ 0.4 & 0.5 & 0.6 \\ 0.7 & 0.8 & 0.9 \end{bmatrix}$$

$$W_V = \begin{bmatrix} 0.7 & 0.8 & 0.9 \\ 1.0 & 1.1 & 1.2 \\ 1.3 & 1.4 & 1.5 \end{bmatrix}$$

Key Matrix

$$W_K$$

- The Key matrix learns weights during training that transform token embeddings into key vectors, which encapsulate the distinguishing features of each token for comparison purposes. This process aids the model in assessing the relevance of each word, such as “bank” or “was,” to another word like “flooded,” within the context of the sentence “The bank of the river was flooded.”

Purpose: It enriches the embeddings, giving them a more meaningful representation. In our case the embeddings are represented by 1 x 3 matrix. Transforming them through the 3 x 3 Key matrix allows us to capture more information. This modification enhances the richness and detail of the data, enabling deeper analytical insights.

Example Calculation:

Given Input embedding:

$$X = [x_1 \quad x_2 \quad x_3]$$

Key matrix

$$W_K = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \end{bmatrix}$$

The resulting **key vector** would be:

$$X \cdot W_K = [x_1 \quad x_2 \quad x_3] \cdot \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \end{bmatrix} = [k_1 \quad k_2 \quad k_3]$$

Where:

$$k_1 = x_1 \cdot w_{11} + x_2 \cdot w_{21} + x_3 \cdot w_{31}$$

$$k_2 = x_1 \cdot w_{12} + x_2 \cdot w_{22} + x_3 \cdot w_{32}$$

$$k_3 = x_1 \cdot w_{13} + x_2 \cdot w_{23} + x_3 \cdot w_{33}$$

- As you can see we have enriched the embeddings with additional data.
- Key vectors (K) capture meaningful features of the tokens that are later useful for the attention mechanism, to calculate attention scores.
- This transformation provides capturing the contextual relationships between tokens during the attention mechanism.
- Instead of using only the original embedding as input, transforming the token through the Key matrix provides additional information, enriching the representation of the token.

Forming the Key Vectors

Now we create this **key vector** for every single word in the sentence.

For the token "[CLS]":

$$\text{CLS} = [1.0 \quad 0.2 \quad 0.1]$$

Calculation:

$$K_{\text{CLS}} = [1.0 \quad 0.2 \quad 0.1] \cdot \begin{bmatrix} 0.1 & 0.2 & 0.3 \\ 0.4 & 0.5 & 0.6 \\ 0.7 & 0.8 & 0.9 \end{bmatrix} = [0.25 \quad 0.38 \quad 0.51]$$

For the token "The":

$$\text{The} = [0.2 \quad 0.8 \quad 0.1]$$

Calculation:

$$K_{\text{The}} = [0.2 \quad 0.8 \quad 0.1] \cdot \begin{bmatrix} 0.1 & 0.2 & 0.3 \\ 0.4 & 0.5 & 0.6 \\ 0.7 & 0.8 & 0.9 \end{bmatrix} = [0.41 \quad 0.52 \quad 0.63]$$

For the token "bank":

$$\text{bank} = [0.2 \quad 0.1 \quad 0.9]$$

Calculation:

$$K_{\text{bank}} = [0.2 \quad 0.1 \quad 0.9] \cdot \begin{bmatrix} 0.1 & 0.2 & 0.3 \\ 0.4 & 0.5 & 0.6 \\ 0.7 & 0.8 & 0.9 \end{bmatrix} = [0.69 \quad 0.81 \quad 0.93]$$

For the token "of":

$$\text{of} = [0.9 \quad 1.0 \quad 0.2]$$

Calculation:

$$K_{\text{of}} = [0.9 \quad 1.0 \quad 0.2] \cdot \begin{bmatrix} 0.1 & 0.2 & 0.3 \\ 0.4 & 0.5 & 0.6 \\ 0.7 & 0.8 & 0.9 \end{bmatrix} = [0.63 \quad 0.84 \quad 1.05]$$

For the token "the" :

$$\text{the} = \begin{bmatrix} 0.2 & 0.8 & 0.1 \end{bmatrix}$$

Calculation:

$$K_{\text{the}} = \begin{bmatrix} 0.2 & 0.8 & 0.1 \end{bmatrix} \cdot \begin{bmatrix} 0.1 & 0.2 & 0.3 \\ 0.4 & 0.5 & 0.6 \\ 0.7 & 0.8 & 0.9 \end{bmatrix} = \begin{bmatrix} 0.41 & 0.52 & 0.63 \end{bmatrix}$$

For the token “river”:

$$\text{river} = \begin{bmatrix} 0.0 & 0.9 & 0.9 \end{bmatrix}$$

Calculation:

$$K_{\text{river}} = \begin{bmatrix} 0.0 & 0.9 & 0.9 \end{bmatrix} \cdot \begin{bmatrix} 0.1 & 0.2 & 0.3 \\ 0.4 & 0.5 & 0.6 \\ 0.7 & 0.8 & 0.9 \end{bmatrix} = \begin{bmatrix} 0.99 & 1.17 & 1.35 \end{bmatrix}$$

For the token “was”:

$$\text{was} = \begin{bmatrix} 0.3 & 0.8 & 0.9 \end{bmatrix}$$

Calculation:

$$K_{\text{was}} = \begin{bmatrix} 0.3 & 0.8 & 0.9 \end{bmatrix} \cdot \begin{bmatrix} 0.1 & 0.2 & 0.3 \\ 0.4 & 0.5 & 0.6 \\ 0.7 & 0.8 & 0.9 \end{bmatrix} = \begin{bmatrix} 0.98 & 1.18 & 1.38 \end{bmatrix}$$

For the token “flooded”:

$$\text{flooded} = \begin{bmatrix} 0.1 & 0.9 & 0.8 \end{bmatrix}$$

Calculation:

$$K_{\text{flooded}} = \begin{bmatrix} 0.1 & 0.9 & 0.8 \end{bmatrix} \cdot \begin{bmatrix} 0.1 & 0.2 & 0.3 \\ 0.4 & 0.5 & 0.6 \\ 0.7 & 0.8 & 0.9 \end{bmatrix} = \begin{bmatrix} 0.93 & 1.11 & 1.29 \end{bmatrix}$$

- For the token “[SEP]”:

$$[\text{SEP}] = \begin{bmatrix} 0.0 & 0.3 & 0.1 \end{bmatrix}$$

Calculation:

$$K_{[\text{SEP}]} = \begin{bmatrix} 0.0 & 0.3 & 0.1 \end{bmatrix} \cdot \begin{bmatrix} 0.1 & 0.2 & 0.3 \\ 0.4 & 0.5 & 0.6 \\ 0.7 & 0.8 & 0.9 \end{bmatrix} = \begin{bmatrix} 0.19 & 0.23 & 0.27 \end{bmatrix}$$

Combine all these Key vectors into a matrix that shows the key vectors:

$$K = \begin{bmatrix} 0.25 & 0.38 & 0.51 \\ 0.41 & 0.52 & 0.63 \\ 0.69 & 0.81 & 0.93 \\ 0.63 & 0.84 & 1.05 \\ 0.41 & 0.52 & 0.63 \\ 0.99 & 1.17 & 1.35 \\ 0.98 & 1.18 & 1.38 \\ 0.93 & 1.11 & 1.29 \\ 0.19 & 0.23 & 0.27 \end{bmatrix}$$

Query Matrix

The similarity between the query and key vectors determines the attention scores, indicating how much focus should be given to various parts of the input sequence.

The Key vectors are used in conjunction with Query vectors to compute attention scores. This comparison helps the model decide how much focus each token should give to every other token in the sequence.

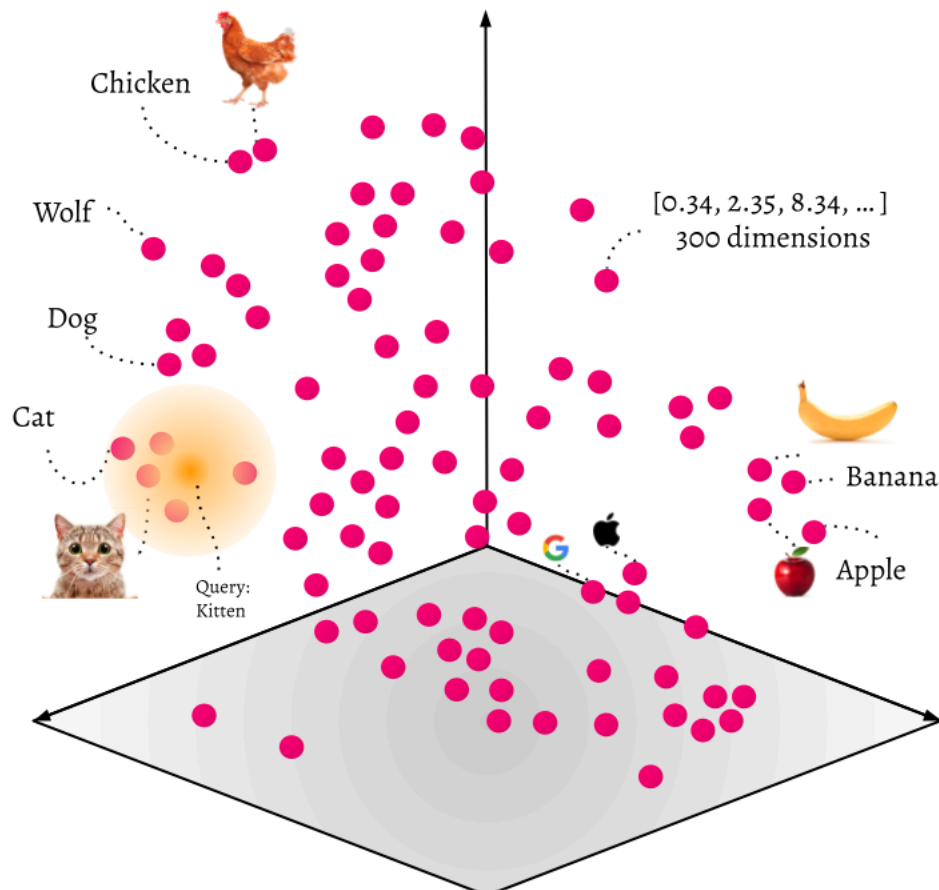
- The query matrix learns parameters so that when multiplied with key vectors it could give meaningful attention scores.
- The Query matrix learns to identify similar or relevant words(to your current word's query vector) within the context of a sentence, helping each token find and focus on the most important relationships. It does this by learning weights that transform token embeddings into Query vectors. These Query vectors are then multiplied by the Key vectors to calculate attention scores, which determine how much the specific word should be influenced by the other words.

Function: The Query matrix transforms the input embeddings into a query representation. This is done to facilitate the comparison between the query and the key representations.

Purpose: It learns parameters that help project the embeddings into a space where the relevance of different elements can be assessed. In essence, the query matrix helps in determining which parts of the input sequence the model should focus on when processing a specific element.

- You find the query vector of a word you are looking for. Then you Calculate the dot product of Query (Q) with each row in the Key matrix K to get the attention scores.

Vector Database Comparison



Reminder this is our Query Matrix (Q)

$$Q = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1.533 & 0 \\ 0 & 0 & -1.311 \end{bmatrix}$$

Given the embedding for “river” is $[[0.0, 0.9, 0.9]]$, the query vector (q) is calculated as:

Compute the Query Vector for “river”

$$q_r = Q \cdot \text{river_embedding} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 15.33 & 0 \\ 0 & 0 & -13.11 \end{bmatrix} \cdot \begin{bmatrix} 0.0 \\ 0.9 \\ 0.9 \end{bmatrix} = \begin{bmatrix} 0 \\ 15.33 \times 0.9 \\ -13.11 \times 0.9 \end{bmatrix} = \begin{bmatrix} 0 \\ 13.80 \\ -11.80 \end{bmatrix}$$

Calculating Raw Attention Scores

Given the query vector (q = $[0.0, 13.80, -11.80]$) and the key matrix (K):

$$K = \begin{bmatrix} 0.25 & 0.38 & 0.51 \\ 0.41 & 0.52 & 0.63 \\ 0.69 & 0.81 & 0.93 \\ 0.63 & 0.84 & 1.05 \\ 0.41 & 0.52 & 0.63 \\ 0.99 & 1.17 & 1.35 \\ 0.98 & 1.18 & 1.38 \\ 0.93 & 1.11 & 1.29 \\ 0.19 & 0.23 & 0.27 \end{bmatrix}$$

We calculate the raw attention scores by performing the dot product of (K) with (q):

$$\text{raw_scores} = K \cdot q = \begin{bmatrix} 0.25 & 0.38 & 0.51 \\ 0.41 & 0.52 & 0.63 \\ 0.69 & 0.81 & 0.93 \\ 0.63 & 0.84 & 1.05 \\ 0.41 & 0.52 & 0.63 \\ 0.99 & 1.17 & 1.35 \\ 0.98 & 1.18 & 1.38 \\ 0.93 & 1.11 & 1.29 \\ 0.19 & 0.23 & 0.27 \end{bmatrix} \cdot \begin{bmatrix} 0 \\ 13.80 \\ -11.80 \end{bmatrix}$$

The individual dot products for each row are calculated as follows:

These below are the raw attention scores(for the word river)

- Scores:

1. ([CLS]):

$$(0.25 \cdot 0) + (0.38 \cdot 13.80) + (0.51 \cdot -11.80) = 0 + 5.244 - 6.018 = -0.774$$

2. (The):

$$(0.41 \cdot 0) + (0.52 \cdot 13.80) + (0.63 \cdot -11.80) = 0 + 7.176 - 7.434 = -0.258$$

3. (Bank):

$$(0.69 \cdot 0) + (0.81 \cdot 13.80) + (0.93 \cdot -11.80) = 0 + 11.178 - 10.974 = 0.204$$

4. (Of):

$$(0.63 \cdot 0) + (0.84 \cdot 13.80) + (1.05 \cdot -11.80) = 0 + 11.592 - 12.39 = -0.798$$

5. (the):

$$(0.41 \cdot 0) + (0.52 \cdot 13.80) + (0.63 \cdot -11.80) = 0 + 7.176 - 7.434 = -0.258$$

6. (River):

$$(0.99 \cdot 0) + (1.17 \cdot 13.80) + (1.35 \cdot -11.80) = 0 + 16.146 - 15.93 = 0.216$$

7. (Was):

$$(0.98 \cdot 0) + (1.18 \cdot 13.80) + (1.38 \cdot -11.80) = 0 + 16.284 - 16.284 \approx 0$$

8. (Flooded):

$$(0.93 \cdot 0) + (1.11 \cdot 13.80) + (1.29 \cdot -11.80) = 0 + 15.318 - 15.222 = 0.096$$

9. ([SEP]):

$$(0.19 \cdot 0) + (0.23 \cdot 13.80) + (0.27 \cdot -11.80) = 0 + 3.174 - 3.186 = -0.012$$

- attention score will be high when we multiply a word's own query vector with its own key vector, since it needs to retain information about itself, not just how it interacts with other words.

Summary of Raw Attention Scores when the query vector is river:

1. ([CLS]): (-0.774)
2. (The): (-0.258)
3. (Bank): (0.204)
4. (Of): (-0.798)
5. (the): (-0.258)
6. (River): (0.216)
7. (Was): (-0.00002)
8. (Flooded): (0.096)
9. ([SEP]): (-0.012)

Summary

The final query matrix (Q), query vector (q) for "river", and the raw attention scores are:

$$Q = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 15.33 & 0 \\ 0 & 0 & -13.11 \end{bmatrix}$$

$$q = [0.0, 13.80, -11.80]$$

Raw Attention Scores:

1. ([CLS]): (-0.774)
2. (The): (-0.258)
3. (Bank): (0.204)
4. (Of): (-0.798)
5. (the): (-0.258)
6. (River): (0.216)
7. (Was): Approximately (-0.00002)
8. (Flooded): (0.096)
9. ([SEP]): (-0.012)

Applying Softmax to Scores

- Convert scores to weights using softmax:
- **Applying Softmax to Raw Attention Scores**

The softmax function is defined as:

$$\text{softmax}(s_i) = \frac{e^{s_i}}{\sum_j e^{s_j}}$$

Given the raw scores:

- ([CLS]): (-0.774)
- (The): (-0.258)
- (Bank): (0.204)
- (Of): (-0.798)
- (the): (-0.258)
- (River): (0.216)
- (Was): (-0.00002)
- (Flooded): (0.096)
- ([SEP]): (-0.012)

Step-by-Step Calculation:

1. **Compute the exponentials of each score:**

$$e^{-0.774}, e^{-0.258}, e^{0.204}, e^{-0.798}, e^{-0.258}, e^{0.216}, e^{-0.00002}, e^{0.096}, e^{-0.012}$$

2. **Sum all exponentials:**

$$\sum_i e^{s_i}$$

3. **Divide each exponential by the sum to get the normalized weights:**

$$\text{softmax}(s_i) = \frac{e^{s_i}}{\sum_j e^{s_j}}$$

Computed Softmax Values :

First, the sum of all exponentials of the raw scores is calculated:

$$\sum e^{s_i} \approx 8.0128$$

Using this sum, the softmax scores are computed as follows:

- $\text{softmax}([\text{CLS}]) = \frac{e^{-0.774}}{8.0128} \approx 0.0576$
- $\text{softmax}(\text{The}) = \frac{e^{-0.258}}{8.0128} \approx 0.0964$
- $\text{softmax}(\text{Bank}) = \frac{e^{0.204}}{8.0128} \approx 0.1530$
- $\text{softmax}(\text{Of}) = \frac{e^{-0.798}}{8.0128} \approx 0.0562$
- $\text{softmax}(\text{the}) = \frac{e^{-0.258}}{8.0128} \approx 0.0964$
- $\text{softmax}(\text{River}) = \frac{e^{0.216}}{8.0128} \approx 0.1549$

•

overall

$$\text{softmax}(\text{Was}) = \frac{e^{-0.00002}}{8.0128} \approx 0.1248$$

•

$$\text{softmax}(\text{Flooded}) = \frac{e^{0.096}}{8.0128} \approx 0.1374$$

•

$$\text{softmax}([\text{SEP}]) = \frac{e^{-0.012}}{8.0128} \approx 0.1233$$

These values represent the attention weights for each token, influencing how each token's value vector contributes to the overall output of the attention layer.

These values will be used as weights for aggregating value vectors in the attention mechanism.

Array of Final Softmax Weights

The softmax weights for each token after normalization are presented in the following array:

[0.0576 0.0964 0.1530 0.0562 0.0964 0.1549 0.1248 0.1374 0.1233]

Each element in the array corresponds to the softmax-calculated weight for each token's attention score, used to modulate the influence of each token's value vector in the output of the attention layer.

Value Matrix

Function: Value vectors contain the detailed information of each token. For "river", the Value vector will have information that captures its characteristics as a geographical feature.

Purpose: The value vectors are used to produce the final output of the attention mechanism. Essentially, these are the transformed embeddings that are weighted and summed based on the attention scores derived from the query-key similarity.

- In essence, the value vector carries the "potential meanings" of a word, while the attention scores determine which of these meanings are most relevant in a given context. This combination allows the model to produce contextually appropriate outputs even when faced with words that have multiple meanings.
- In our example with "bank" and "river," the standalone meaning of a word like "bank" might be ambiguous(riverbank or financial bank) or not fully informative. In such cases, the attention mechanism supplements the information in the value vector by focusing on the context provided by other words (like "river"). The attention scores determine how much each word's value vector should influence the final representation.
- In summary, the value vector provides a foundation of knowledge about a word, and the attention mechanism refines this knowledge, enhancing it with contextual information to produce a nuanced and accurate interpretation of text.

Putting It All Together

Given the embeddings and the value matrix:

Embeddings:

- [CLS]: ([1.0, 0.2, 0.1])
- The: ([0.2, 0.8, 0.1])
- Bank: ([0.2, 0.1, 0.9])
- Of: ([0.9, 1.0, 0.2])
- the: ([0.2, 0.8, 0.1])
- River: ([0.0, 0.9, 0.9])

- Was: ([0.3, 0.8, 0.9])
- Flooded: ([0.1, 0.9, 0.8])
- [SEP]: ([0.0, 0.3, 0.1])

Value Matrix :

$$W_V = \begin{bmatrix} 0.7 & 0.8 & 0.9 \\ 1.0 & 1.1 & 1.2 \\ 1.3 & 1.4 & 1.5 \end{bmatrix}$$

Calculations:

Each value vector is computed by multiplying the corresponding embedding vector by the value matrix

- For [CLS]:

$$[\text{CLS}] = \begin{bmatrix} 1.0 & 0.2 & 0.1 \end{bmatrix} \cdot \begin{bmatrix} 0.7 & 0.8 & 0.9 \\ 1.0 & 1.1 & 1.2 \\ 1.3 & 1.4 & 1.5 \end{bmatrix} = \begin{bmatrix} 1.03 & 1.16 & 1.29 \end{bmatrix}$$

- For The:

$$\text{The} = \begin{bmatrix} 0.2 & 0.8 & 0.1 \end{bmatrix} \cdot \begin{bmatrix} 0.7 & 0.8 & 0.9 \\ 1.0 & 1.1 & 1.2 \\ 1.3 & 1.4 & 1.5 \end{bmatrix} = \begin{bmatrix} 1.07 & 1.18 & 1.29 \end{bmatrix}$$

- For Bank:

$$\text{Bank} = \begin{bmatrix} 0.2 & 0.1 & 0.9 \end{bmatrix} \cdot \begin{bmatrix} 0.7 & 0.8 & 0.9 \\ 1.0 & 1.1 & 1.2 \\ 1.3 & 1.4 & 1.5 \end{bmatrix} = \begin{bmatrix} 1.41 & 1.53 & 1.65 \end{bmatrix}$$

- For Of:

$$\text{Of} = \begin{bmatrix} 0.9 & 1.0 & 0.2 \end{bmatrix} \cdot \begin{bmatrix} 0.7 & 0.8 & 0.9 \\ 1.0 & 1.1 & 1.2 \\ 1.3 & 1.4 & 1.5 \end{bmatrix} = \begin{bmatrix} 1.89 & 2.1 & 2.31 \end{bmatrix}$$

- For the:

$$\text{the} = \begin{bmatrix} 0.2 & 0.8 & 0.1 \end{bmatrix} \cdot \begin{bmatrix} 0.7 & 0.8 & 0.9 \\ 1.0 & 1.1 & 1.2 \\ 1.3 & 1.4 & 1.5 \end{bmatrix} = \begin{bmatrix} 1.07 & 1.18 & 1.29 \end{bmatrix}$$

- For River:

$$\text{River} = \begin{bmatrix} 0.0 & 0.9 & 0.9 \end{bmatrix} \cdot \begin{bmatrix} 0.7 & 0.8 & 0.9 \\ 1.0 & 1.1 & 1.2 \\ 1.3 & 1.4 & 1.5 \end{bmatrix} = \begin{bmatrix} 2.07 & 2.25 & 2.43 \end{bmatrix}$$

- For Was:

$$\text{Was} = \begin{bmatrix} 0.3 & 0.8 & 0.9 \end{bmatrix} \cdot \begin{bmatrix} 0.7 & 0.8 & 0.9 \\ 1.0 & 1.1 & 1.2 \\ 1.3 & 1.4 & 1.5 \end{bmatrix} = \begin{bmatrix} 2.18 & 2.38 & 2.58 \end{bmatrix}$$

- For Flooded:

$$\text{Flooded} = \begin{bmatrix} 0.1 & 0.9 & 0.8 \end{bmatrix} \cdot \begin{bmatrix} 0.7 & 0.8 & 0.9 \\ 1.0 & 1.1 & 1.2 \\ 1.3 & 1.4 & 1.5 \end{bmatrix} = \begin{bmatrix} 2.01 & 2.19 & 2.37 \end{bmatrix}$$

- For [SEP]:

$$[\text{SEP}] = \begin{bmatrix} 0.0 & 0.3 & 0.1 \end{bmatrix} \cdot \begin{matrix} \text{overall} \\ \begin{bmatrix} 0.7 & 0.8 & 0.9 \\ 1.0 & 1.1 & 1.2 \\ 1.3 & 1.4 & 1.5 \end{bmatrix} \end{matrix} = \begin{bmatrix} 0.43 & 0.47 & 0.51 \end{bmatrix}$$

- Value vector shows the meaning of the word, but it needs to know which words it should focus on. Like in this case it doesn't have enough info about a word like bank, so it uses the attention mechanism to supplement its knowledge.

Applying Softmax Weights to Value Vectors

- this is the part where we add attention to the value vectors

Given the computed softmax weights and the value vectors:

Softmax Weights:

- CLS: (0.0576)
- The: (0.0964)
- Bank: (0.1530)
- Of: (0.0562)
- the: (0.0964)
- River: (0.1549)
- Was: (0.1248)
- Flooded: (0.1374)
- SEP: (0.1233)

Value Vectors:

- [CLS]: ([1.03, 1.16, 1.29])
- The: ([1.07, 1.18, 1.29])
- Bank: ([1.41, 1.53, 1.65])
- Of: ([1.89, 2.1, 2.31])
- the: ([1.07, 1.18, 1.29])
- River: ([2.07, 2.25, 2.43])
- Was: ([2.18, 2.38, 2.58])
- Flooded: ([2.01, 2.19, 2.37])
- [SEP]: ([0.43, 0.47, 0.51])

Multiply Each Value Vector by Its Corresponding Softmax Weight:

$$\text{Weighted [CLS]} : 0.0576 \cdot [1.03, 1.16, 1.29] = [0.0593, 0.0668, 0.0743]$$

$$\text{Weighted The} : 0.0964 \cdot [1.07, 1.18, 1.29] = [0.1031, 0.1138, 0.1245]$$

$$\text{Weighted Bank} : 0.1530 \cdot [1.41, 1.53, 1.65] = [0.2158, 0.2340, 0.2525]$$

$$\text{Weighted Of} : 0.0562 \cdot [1.89, 2.1, 2.31] = [0.1062, 0.1180, 0.1298]$$

$$\text{Weighted the (second)} : 0.0964 \cdot [1.07, 1.18, 1.29] = [0.1031, 0.1138, 0.1245]$$

$$\text{Weighted River} : 0.1549 \cdot [2.07, 2.25, 2.43] = [0.3205, 0.3486, 0.3766]$$

$$\text{Weighted Was} : 0.1248 \cdot [2.18, 2.38, 2.58] = [0.2721, 0.2970, 0.3219]$$

$$\text{Weighted Flooded} : 0.1374 \cdot [2.01, 2.19, 2.37] = [0.2763, 0.3010, 0.3257]$$

$$\text{Weighted [SEP]} : 0.1233 \cdot [0.43, 0.47, 0.51] = [0.0530, 0.0580, 0.0629]$$

Summing the Weighted Value Vectors to Form a Context-Aware Representation for "River"

After calculating the weighted value vectors for each token based on the softmax weights, the next step is to aggregate these vectors to produce a single, context-enriched representation of the query word “river”. This is achieved by summing the weighted value vectors:

Formula for Aggregation:

$$\text{Context Vector for River} = \sum_{i=1}^n \text{Weighted Value Vector}_i$$

Where (n) is the number of tokens in the sequence, and Weighted Value Vector is the weighted value vector for the (i)-th token.

Summing Weighted Value Vectors for “River”

The final context-aware representation for “River” is calculated by summing all the weighted value vectors for each token in the input sequence:

Sum the Weighted Value Vectors

Output vector for ”River” = [0.0593, 0.0668, 0.0743] + [0.1031, 0.1138, 0.1245] + [0.2158, 0.2340, 0.2525] + [0.106

Final vector for ”river” = [1.5094, 1.6410, 1.7726]

(in a real attention mechanism you would find the query vector for all values not just river)

This calculation aggregates the individual contributions of all tokens in the sequence, weighted by their relevance to “River” as determined by the attention mechanism, resulting in a comprehensive context vector.

This result reflects the aggregated influences from all tokens weighted by their significance to the context of “River” as computed by the attention mechanism.

This computation aggregates the weighted vectors for each token into a single vector for “River,” capturing contextual influences from the entire sequence.

Interpretation:

The resulting context vector for “River” encapsulates the influence of all other words in the sequence, giving a nuanced representation that goes beyond its standalone meaning to include contextual nuances influenced by the entire sentence or sequence. This vector can then be used for further processing in the neural network, such as in downstream tasks like classification, translation, or other forms of language understanding and generation.

What Happens in real life:

1. Multiple Parallel Attention Heads:

- The input sentence (sequence of token embeddings) is processed by multiple (e.g., 12) attention heads in parallel.
- Each attention head computes a separate attention mechanism independently but using the same input token embeddings.
- Each attention head has its own set of Query, Key and Value Matrices
- Parallel Operation: All 12 attention heads within a layer process the input data at the same time, independently from each other.
- for our example for simplicity we only have a single head, and no multi head attention.

- **Focus Areas for Groups of Attention Heads**

- **Heads 1-12 (Layer 1)** - Focus on basic syntax and grammar: Capturing essential grammatical structures, such as subject-verb agreement, tense, and basic sentence parsing.
- **Heads 13-24 (Layer 2)** - Address lexical relationships: Analyze the relationships between nouns, verbs, and adjectives, understanding how they interact to convey meaning within sentences.
- **Heads 25-36 (Layer 3)** - Enhance contextual understanding: Dive deeper into the context of phrases and clauses, recognizing the relevance and implications of the surrounding text.
- **Heads 37-48 (Layer 4)** - Process semantic roles and dependencies: Identify and interpret the roles entities play in a sentence, understanding complex dependencies and semantic relationships.
- **Heads 49-60 (Layer 5)** - Manage discourse and narrative flow: Analyze how elements like conjunctions, discourse markers, and overall text structure guide the logical flow and coherence of paragraphs and larger text bodies.
- **Heads 61-72 (Layer 6)** - Specialized analysis and integration: Tackle specialized topics, pragmatic cues, emotional tone, and potentially multimodal data integration (if extended beyond the original model capabilities). This can include sentiment analysis, figurative language interpretation, and complex intertextual connections.
- **Output of Attention Heads:** Each attention head generates a context-aware vector for each token in the input sequence.
- **Combining Outputs:** Since there are 12 heads in each transformer layer of DistilBERT, this means for each token, you end up with 12 different vectors from each layer.

3. Concatenation:

- This longer vector now encapsulates the information from all 12 attention heads.
- Multi-head attention consists of multiple single attention heads operating in parallel. Each head independently computes its attention output, focusing on different aspects of the input. The outputs from all heads are then concatenated into a single vector and passed through a final linear transformation, allowing the model to capture diverse information.
- Example :

Attention Head 1: Output vector for "bank" $\rightarrow \begin{bmatrix} 0.1 \\ 0.2 \\ 0.3 \end{bmatrix}$

Attention Head 2: Output vector for "bank" $\rightarrow \begin{bmatrix} 0.4 \\ 0.5 \\ 0.6 \end{bmatrix}$

Attention Head 3: Output vector for "bank" $\rightarrow \begin{bmatrix} 0.7 \\ 0.8 \\ 0.9 \end{bmatrix}$

Attention Head 4: Output vector for "bank" $\rightarrow \begin{bmatrix} 1.0 \\ 1.1 \\ 1.2 \end{bmatrix}$

Attention Head 5: Output vector for "bank" $\rightarrow \begin{bmatrix} 1.3 \\ 1.4 \\ 1.5 \end{bmatrix}$

$$\begin{aligned}
 \text{Attention Head 6: Output vector for "bank"} &\rightarrow \begin{bmatrix} 1.6 \\ 1.7 \\ 1.8 \end{bmatrix} \\
 \text{Attention Head 7: Output vector for "bank"} &\rightarrow \begin{bmatrix} 1.9 \\ 2.0 \\ 2.1 \end{bmatrix} \\
 \text{Attention Head 8: Output vector for "bank"} &\rightarrow \begin{bmatrix} 2.2 \\ 2.3 \\ 2.4 \end{bmatrix} \\
 \text{Attention Head 9: Output vector for "bank"} &\rightarrow \begin{bmatrix} 2.5 \\ 2.6 \\ 2.7 \end{bmatrix} \\
 \text{Attention Head 10: Output vector for "bank"} &\rightarrow \begin{bmatrix} 2.8 \\ 2.9 \\ 3.0 \end{bmatrix} \\
 \text{Attention Head 11: Output vector for "bank"} &\rightarrow \begin{bmatrix} 3.1 \\ 3.2 \\ 3.3 \end{bmatrix} \\
 \text{Attention Head 12: Output vector for "bank"} &\rightarrow \begin{bmatrix} 3.4 \\ 3.5 \\ 3.6 \end{bmatrix}
 \end{aligned}$$

$$\text{Concatenated vector for "bank"} = [0.1 \ 0.2 \ 0.3 \ 0.4 \ 0.5 \ 0.6 \ 0.7 \ 0.8 \ 0.9 \ 1.0 \ 1.1 \ 1.2 \ 1.3 \ 1.4 \ 1.5]$$

The output vectors you see listed for each of the 12 attention heads would represent the context-aware vectors for the token "bank" generated within one single transformer layer of DistilBERT. The concatenated vector at the end is the combination of all these individual vectors from the 12 heads in that layer, forming a richer, multidimensional representation of the token. This concatenated vector then feeds into subsequent layers or is used in further processing tasks within the model.

After concatenation:

- **Transformation:** The concatenated vector for each token then passes through a feed-forward neural network within the same transformer layer. This network usually consists of two linear transformations with a ReLU activation in between. It's designed to further process the information and integrate the features extracted by the attention mechanism.
- **Residual Connection:** The output of the feed-forward network is added back to the input of the layer (before the attention heads) via a residual connection. This helps in combating the vanishing gradient problem by allowing gradients to flow directly through the network.
- **Layer Normalization:** After the residual connection, layer normalization is applied. This step normalizes the vector to have zero mean and unit variance, helping to stabilize the learning process.

5. Output to Next Layer

The output from this entire process (post-normalization) forms the input to the next transformer layer. It carries the refined and contextually enriched representations of each token, including "bank", to undergo further processing in subsequent layers.